SRI CHANDRASEKHADRENDRA SARASWATHI VISWA MAHAVIDYALAYA

UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1G56) ENATHUR, KANCHIPURAM - 631 561

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Name : P. Sri Manaswini

Reg No : 11249A356

Class : III B.E(CSE)

Course Code : BPSF185T40

Course Name : Database Management Systems

SRI CHANDRASEKHADRENDRA SARASWATHI VISWA MAHAVIDYALAYA

UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1G56) ENATHUR, KANCHIPURAM - 631 561

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

BONAFIDE CERTIFICATE

This is to certify that this is the Bonafide record of work done by

Ms. P. Sri Manaswini

with Reg. No 11249A356 done by of III-B.E.(CSE) in Database Management Systems Lab during the academic year 2026 -2027.

Station : Enathur

Date :5-11-2026

Staff-in-charge

Head of the department

Submitted for the Practical examination held on 11-11-2026

Examine-1

Examine

Index

S.NO	CONTENT	PG.NO
1	Creating a Database with Commands	4
2	Creating a Database Table with Various Constraints	8
3	Set Operators and Subqueries	11
4	Aggregate Functions	14
5	Joins	15
6	Explicit Cursor in PL/SQL	18
7	Trigger	20
8	PL/SQL Block for Exception Handling	22
9	Stored Procedure	24
10	Stored Function	26
11	Hospital Management System	28
12	Hotel Management System	30
13	Timetable Management System	33
14	Railway Reservation System	35

1. Creating a Database with Commands

Aim

To practise database/schema setup concepts and common SQL commands such as CREATE, INSERT, SELECT, WHERE, ORDER BY, UPDATE, DELETE, ALTER, CREATE TABLE AS SELECT, TRUNCATE, DROP, and COMMIT.

Algorithm

1. Connect to the Oracle schema assigned for the DBMS lab.
2. If permitted, create a user and grant the required privileges; otherwise use the existing schema.
3. Create the Student table with columns for ID, name, department, and marks.
4. Insert three sample student records and commit the changes.
5. Display all records and selected columns using SELECT.
6. Filter CSE students using WHERE and sort marks using ORDER BY.
7. Update a student's marks and verify the changed value.
8. Delete one sample row and verify the remaining records.
9. Add an email column with ALTER TABLE and populate it.
10. Rename the department column and create a copy table using CREATE TABLE AS SELECT.
11. Run TRUNCATE and DROP on the copy table, then display the final Student table.

SQL Program

```
-- PROGRAM 1: DATABASE / SCHEMA AND BASIC SQL COMMANDS
-- Oracle note: CREATE DATABASE is normally performed by a DBA.
-- In a student lab, connect to your assigned schema and run the commands below.
-- Optional DBA commands (run only if you have administrator permission):
-- CREATE USER dbms_student IDENTIFIED BY student123;
-- GRANT CREATE SESSION, CREATE TABLE, CREATE VIEW, CREATE SEQUENCE,
--       CREATE PROCEDURE, CREATE TRIGGER TO dbms_student;
-- CONNECT dbms_student/student123;

-- 1. Create a table
CREATE TABLE Student (
    student_id NUMBER(4) PRIMARY KEY,
    student_name VARCHAR2(30),
    department VARCHAR2(10),
    marks NUMBER(3)
);

-- 2. Insert records
INSERT INTO Student VALUES (1, 'Anu', 'CSE', 85);
INSERT INTO Student VALUES (2, 'Bala', 'ECE', 78);
INSERT INTO Student VALUES (3, 'Charan', 'CSE', 92);

-- 3. Save changes
COMMIT;

-- 4. Display all records
SELECT * FROM Student ORDER BY student_id;

-- 5. Display selected columns
SELECT student_name, marks FROM Student ORDER BY student_id;

-- 6. Filter records using WHERE
SELECT * FROM Student WHERE department = 'CSE';
```

```
-- 7. Sort records using ORDER BY
SELECT * FROM Student ORDER BY marks DESC;

-- 8. Update a record
UPDATE Student SET marks = 88 WHERE student_id = 1;
SELECT * FROM Student WHERE student_id = 1;

-- 9. Delete one record (demonstration)
DELETE FROM Student WHERE student_id = 3;
SELECT * FROM Student ORDER BY student_id;

-- 10. Add a new column using ALTER TABLE
ALTER TABLE Student ADD email VARCHAR2(50);
UPDATE Student SET email = 'anu@example.com' WHERE student_id = 1;
UPDATE Student SET email = 'bala@example.com' WHERE student_id = 2;

-- 11. Rename a column
ALTER TABLE Student RENAME COLUMN department TO dept_name;

-- 12. Create another table using CREATE TABLE AS SELECT
CREATE TABLE CSE_Students AS
SELECT student_id, student_name, marks
FROM Student
WHERE dept_name = 'CSE';

SELECT * FROM CSE_Students;

-- 13. Remove all rows from the copied table (keeps table structure)
TRUNCATE TABLE CSE_Students;

-- 14. Remove the copied table
DROP TABLE CSE_Students;

-- 15. View the final Student table
SELECT * FROM Student ORDER BY student_id;

-- Note: ROLLBACK reverses uncommitted DML only. It cannot undo COMMIT,
-- and Oracle DDL statements such as CREATE/ALTER/DROP commit implicitly.
```

Sample Output

```
Initial SELECT * FROM Student
STUDENT_ID  STUDENT_NAME  DEPARTMENT  MARKS
1           Anu           CSE         85
2           Bala          ECE         78
3           Charan        CSE         92

After UPDATE of Anu
STUDENT_ID  STUDENT_NAME  DEPARTMENT  MARKS
1           Anu           CSE         88

After DELETE of Charan, and renaming DEPARTMENT to DEPT_NAME
STUDENT_ID  STUDENT_NAME  DEPT_NAME  MARKS  EMAIL
1           Anu           CSE        88     anu@example.com
2           Bala          ECE        78     bala@example.com

CSE_Students initially contains Anu; after TRUNCATE it contains 0 rows.
After DROP TABLE CSE_Students, that table no longer exists.
```

Result

The program demonstrates table creation, data insertion and retrieval, filtering, sorting, updating, deleting, altering table structure, copying data into a new table, truncating rows, dropping a table, and committing changes.

SQL Plus

```
Prince                                                  75000 22-NOV-20

EMPLOYEEID FIRSTNAME
---------- --------------------------------------------------
LASTNAME                                           SALARY HIREDATE
-------------------------------------------------- ---------- ---------
DEPARTMENTID
------------
           3

       105 Evan
Wright                                                  92000 18-FEB-24
           2


SQL> SELECT FirstName, LastName, Salary FROM Employees;

FIRSTNAME
--------------------------------------------------
LASTNAME                                           SALARY
-------------------------------------------------- ----------
Alice
Smith                                                   85000

Bob
Johnson                                                 95000

Charlie
Brown                                                   60000


FIRSTNAME
--------------------------------------------------
LASTNAME                                           SALARY
-------------------------------------------------- ----------
Diana
Prince                                                  75000

Evan
Wright                                                  92000


SQL> SELECT * FROM Employees
  2  WHERE Salary > 80000.00;
```

2. Creating a Database Table with Various Constraints

Aim

To create tables and apply PRIMARY KEY, NOT NULL, UNIQUE, CHECK, DEFAULT, and FOREIGN KEY constraints, insert valid data, inspect constraints, and understand examples of invalid data that constraints reject.

Algorithm

1. Create the Department parent table with a primary key and a unique, non-null department name.
2. Insert the department rows before creating employee records.
3. Create the Employee table with primary key, NOT NULL, UNIQUE, CHECK, DEFAULT, and FOREIGN KEY constraints.
4. Insert valid employee records that satisfy all constraints.
5. Insert one employee without a salary to demonstrate the DEFAULT value.
6. Display the Employee table and query USER_CONSTRAINTS to verify the constraints.
7. Optionally run each commented invalid INSERT separately to observe the relevant constraint violation.

SQL Program

```
-- PROGRAM 2: TABLE CREATION WITH CONSTRAINTS
-- PRIMARY KEY, NOT NULL, UNIQUE, CHECK, DEFAULT, and FOREIGN KEY

-- Parent table must be created first because Employee references it.
CREATE TABLE Department (
    dept_id NUMBER(3),
    dept_name VARCHAR2(20) CONSTRAINT uq_department_name UNIQUE NOT NULL,
    CONSTRAINT pk_department PRIMARY KEY (dept_id)
);

INSERT INTO Department (dept_id, dept_name) VALUES (10, 'CSE');
INSERT INTO Department (dept_id, dept_name) VALUES (20, 'ECE');
INSERT INTO Department (dept_id, dept_name) VALUES (30, 'MECH');

-- Employee table demonstrates multiple constraints.
CREATE TABLE Employee (
    emp_id NUMBER(4),
    emp_name VARCHAR2(30) CONSTRAINT nn_employee_name NOT NULL,
    email VARCHAR2(50),
    age NUMBER(2),
    salary NUMBER(8,2) DEFAULT 25000,
    dept_id NUMBER(3),
    CONSTRAINT pk_employee PRIMARY KEY (emp_id),
    CONSTRAINT uq_employee_email UNIQUE (email),
    CONSTRAINT chk_employee_age CHECK (age >= 18),
    CONSTRAINT chk_employee_salary CHECK (salary > 0),
    CONSTRAINT fk_employee_department FOREIGN KEY (dept_id)
        REFERENCES Department(dept_id)
);

-- Valid insertions
INSERT INTO Employee (emp_id, emp_name, email, age, salary, dept_id)
VALUES (101, 'Anu', 'anu@example.com', 20, 35000, 10);

INSERT INTO Employee (emp_id, emp_name, email, age, salary, dept_id)
VALUES (102, 'Bala', 'bala@example.com', 21, 42000, 20);

-- DEFAULT constraint: salary is omitted, so Oracle uses 25000.
```

```
INSERT INTO Employee (emp_id, emp_name, email, age, dept_id)
VALUES (103, 'Charan', 'charan@example.com', 19, 10);

COMMIT;

-- Display the inserted records
SELECT * FROM Employee ORDER BY emp_id;

-- Display table constraint definitions
SELECT constraint_name, constraint_type, status
FROM user_constraints
WHERE table_name IN ('DEPARTMENT', 'EMPLOYEE')
ORDER BY table_name, constraint_type;

-- OPTIONAL TESTS: run each separately; each is expected to fail.
-- PRIMARY KEY violation: duplicate emp_id
-- INSERT INTO Employee VALUES (101, 'Duplicate', 'dup@example.com', 20, 30000, 10);

-- NOT NULL violation: emp_name omitted
-- INSERT INTO Employee (emp_id, email, age, salary, dept_id)
-- VALUES (104, 'no_name@example.com', 20, 30000, 10);

-- UNIQUE violation: duplicate email
-- INSERT INTO Employee VALUES (104, 'Divya', 'anu@example.com', 20, 30000, 10);

-- CHECK violation: age below 18
-- INSERT INTO Employee VALUES (105, 'Esha', 'esha@example.com', 16, 30000, 10);

-- CHECK violation: non-positive salary
-- INSERT INTO Employee VALUES (106, 'Farah', 'farah@example.com', 20, -500, 10);

-- FOREIGN KEY violation: dept_id 99 does not exist in Department
-- INSERT INTO Employee VALUES (107, 'Gopi', 'gopi@example.com', 20, 30000, 99);
```

Sample Output

```
EMP_ID  EMP_NAME  EMAIL               AGE  SALARY  DEPT_ID
------  --------  ------------------  ---  ------  -------
101     Anu       anu@example.com     20   35000   10
102     Bala      bala@example.com    21   42000   20
103     Charan    charan@example.com  19   25000   10

The USER_CONSTRAINTS query lists the named constraints and their status (ENABLED).

Optional invalid INSERT statements are commented out because each should raise a constraint error
when run.
```

Result

The Department and Employee tables were created successfully with constraints. Valid records were inserted, the default salary was applied when omitted, and invalid examples demonstrate how constraints protect data integrity.

SQL Plus

```
SQL> INSERT INTO JobRoles (RoleID, MinExperienceYears, EmploymentType) VALUES (1, 5, 'Full-Time');

1 row created.

SQL> INSERT INTO JobRoles (RoleID, MinExperienceYears, EmploymentType) VALUES (2, -3, 'Freelance');
INSERT INTO JobRoles (RoleID, MinExperienceYears, EmploymentType) VALUES (2, -3, 'Freelance')
*
ERROR at line 1:
ORA-02290: check constraint (A367.CHK_EMP_TYPE) violated


SQL> CREATE TABLE SystemLogs (
  2  LogID INT PRIMARY KEY,
  3   LogMessage VARCHAR2(4000),
  4   EntryDate DATE DEFAULT SYSDATE);

Table created.

SQL> INSERT INTO SystemLogs (LogID, LogMessage) VALUES (1, 'System started normally.');

1 row created.

SQL> SELECT * FROM Departments;

DEPARTMENTID DEPARTMENTNAME
------------ --------------------------------------------------
          10 Finance

SQL> SELECT TeamID, TeamName FROM ProjectTeams;

    TEAMID
----------
TEAMNAME
--------------------------------------------------------------------------------
         1
Alpha Team


SQL> SELECT ClientID
  2  FROM ClientAccounts
  3  WHERE CorporateEmail IS NULL;
```

3. Set Operators and Subqueries

Aim

To demonstrate UNION, UNION ALL, INTERSECT, MINUS, and a subquery using sample tables.

Algorithm

1. Create two tables containing student names.
2. Insert sample data into both tables.
3. Run set-operator queries to compare the results.
4. Run a subquery to find students whose marks are above the average.
5. Observe the results.

SQL Program

```
CREATE TABLE Club_A (
    student_name VARCHAR2(30)
);
CREATE TABLE Club_B (
    student_name VARCHAR2(30)
);

INSERT INTO Club_A VALUES ('Anu');
INSERT INTO Club_A VALUES ('Bala');
INSERT INTO Club_A VALUES ('Charan');

INSERT INTO Club_B VALUES ('Bala');
INSERT INTO Club_B VALUES ('Divya');
INSERT INTO Club_B VALUES ('Anu');

CREATE TABLE Student_Marks (
    student_id NUMBER PRIMARY KEY,
    student_name VARCHAR2(30),
    marks NUMBER(3)
);
INSERT INTO Student_Marks VALUES (1, 'Anu', 85);
INSERT INTO Student_Marks VALUES (2, 'Bala', 60);
INSERT INTO Student_Marks VALUES (3, 'Charan', 95);
INSERT INTO Student_Marks VALUES (4, 'Divya', 70);
COMMIT;

-- UNION: distinct names from both tables
SELECT student_name FROM Club_A
UNION
SELECT student_name FROM Club_B
ORDER BY student_name;

-- UNION ALL: includes duplicates
SELECT student_name FROM Club_A
UNION ALL
SELECT student_name FROM Club_B;

-- INTERSECT: names present in both tables
SELECT student_name FROM Club_A
INTERSECT
SELECT student_name FROM Club_B;
```

```
-- MINUS: names in Club_A but not in Club_B
SELECT student_name FROM Club_A
MINUS
SELECT student_name FROM Club_B;

-- Subquery: marks greater than the average
SELECT student_name, marks
FROM Student_Marks
WHERE marks > (SELECT AVG(marks) FROM Student_Marks)
ORDER BY marks;
```

Sample Output

```
UNION
ANU
BALA
CHARAN
DIVYA

INTERSECT
ANU
BALA

MINUS
CHARAN

Subquery (average marks = 77.5)
STUDENT_NAME  MARKS
------------  -----
Anu           85
Charan        95
```

Note: UNION ALL returns all six rows, including repeated Anu and Bala. Each query is run separately, so the displayed headings vary by SQL tool.

Result

Set operators combine or compare query results, and the subquery returns students scoring above the average.

SQL Plus

```
SQL> CREATE TABLE Customers (
  2   CustomerID INT PRIMARY KEY,
  3   CustomerName VARCHAR2(50) NOT NULL,
  4   City VARCHAR2(50));

Table created.

SQL> CREATE TABLE Orders (
  2  OrderID INT PRIMARY KEY,
  3  OrderDate DATE DEFAULT SYSDATE,
  4   TotalAmount NUMBER(10, 2),
  5  CustomerID INT,
  6   CONSTRAINT fk_orders_customer
  7  FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID));

Table created.

SQL> INSERT INTO Customers (CustomerID, CustomerName, City) VALUES (1, 'Alice Smith', 'Mumbai');

1 row created.

SQL> INSERT INTO Customers (CustomerID, CustomerName, City) VALUES (2, 'Bob Jones', 'Delhi');

1 row created.

SQL> INSERT INTO Customers (CustomerID, CustomerName, City) VALUES (3, 'Charlie Brown', 'Mumbai');

1 row created.

SQL> INSERT INTO Orders (OrderID, OrderDate, TotalAmount, CustomerID) VALUES (101, TO_DATE('2026-09-01', 'YYYY-MM-DD'), 1500.00, 1);

1 row created.

SQL> INSERT INTO Orders (OrderID, OrderDate, TotalAmount, CustomerID) VALUES (102, TO_DATE('2026-09-05', 'YYYY-MM-DD'), 3200.00, 1);

1 row created.

SQL> INSERT INTO Orders (OrderID, OrderDate, TotalAmount, CustomerID) VALUES (103, TO_DATE('2026-09-09', 'YYYY-MM-DD'), 450.00, 2);

1 row created.

SQL> SELECT c.CustomerName, o.OrderID, o.TotalAmount
  2  FROM Customers c
  3  INNER JOIN Orders o ON c.CustomerID = o.CustomerID;
```

4. Aggregate Functions

Aim

To use COUNT, SUM, AVG, MAX, and MIN on a table containing student marks.

Algorithm

1. Create the Student_Marks table.
2. Insert sample student marks.
3. Use aggregate functions to calculate count, total, average, highest, and lowest marks.
4. Read the returned values.

SQL Program

```
CREATE TABLE Student_Marks (
    student_id NUMBER PRIMARY KEY,
    student_name VARCHAR2(30),
    marks NUMBER(3)
);

INSERT INTO Student_Marks VALUES (1, 'Anu', 85);
INSERT INTO Student_Marks VALUES (2, 'Bala', 60);
INSERT INTO Student_Marks VALUES (3, 'Charan', 95);
INSERT INTO Student_Marks VALUES (4, 'Divya', 70);
COMMIT;

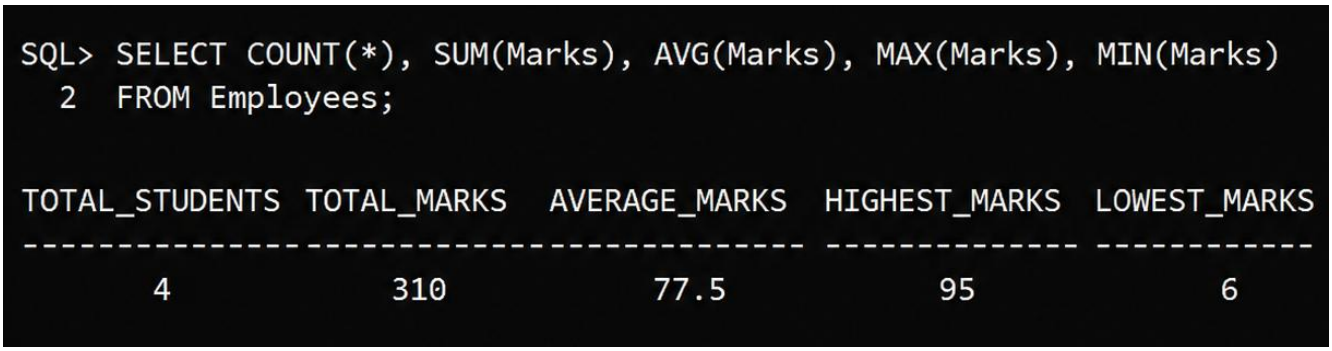
SELECT COUNT(*) AS total_students,
       SUM(marks) AS total_marks,
       ROUND(AVG(marks), 2) AS average_marks,
       MAX(marks) AS highest_marks,
       MIN(marks) AS lowest_marks
FROM Student_Marks;
```

Sample Output

```
TOTAL_STUDENTS  TOTAL_MARKS  AVERAGE_MARKS  HIGHEST_MARKS  LOWEST_MARKS
--------------  -----------  -------------  ---------      ------------
4               310          77.5           95             60
```

Result

Aggregate functions successfully calculate summary values for the marks column.



5. Joins

Aim

To demonstrate INNER JOIN, LEFT OUTER JOIN, and RIGHT OUTER JOIN using Department and Employee tables.

Algorithm

5. Create Department and Employee tables.
6. Insert departments and employees, including one employee without a department assignment.
7. Execute the join queries.
8. Compare the matching and non-matching rows.

SQL Program

```
CREATE TABLE Department (
    dept_id NUMBER PRIMARY KEY,
    dept_name VARCHAR2(20)
);
CREATE TABLE Employee (
    emp_id NUMBER PRIMARY KEY,
    emp_name VARCHAR2(30),
    dept_id NUMBER
);

INSERT INTO Department VALUES (10, 'CSE');
INSERT INTO Department VALUES (20, 'ECE');
INSERT INTO Department VALUES (30, 'MECH');

INSERT INTO Employee VALUES (101, 'Anu', 10);
INSERT INTO Employee VALUES (102, 'Bala', 20);
INSERT INTO Employee VALUES (103, 'Charan', 10);
INSERT INTO Employee VALUES (104, 'Divya', NULL);
COMMIT;

-- INNER JOIN: only matching department rows
SELECT e.emp_name, d.dept_name
FROM Employee e
INNER JOIN Department d ON e.dept_id = d.dept_id
ORDER BY e.emp_id;

-- LEFT JOIN: every employee, including unmatched employees
SELECT e.emp_name, d.dept_name
FROM Employee e
LEFT JOIN Department d ON e.dept_id = d.dept_id
ORDER BY e.emp_id;

-- RIGHT JOIN: every department, including departments with no employees
SELECT e.emp_name, d.dept_name
FROM Employee e
RIGHT JOIN Department d ON e.dept_id = d.dept_id
ORDER BY d.dept_id, e.emp_id;
```

Sample Output

```
INNER JOIN
EMP_NAME  DEPT_NAME
--------  ---------
Anu       CSE
Bala      ECE
Charan    CSE

LEFT JOIN
EMP_NAME  DEPT_NAME
--------  ---------
Anu       CSE
Bala      ECE
Charan    CSE
Divya     (NULL)

RIGHT JOIN
EMP_NAME  DEPT_NAME
--------  ---------
Anu       CSE
Charan    CSE
Bala      ECE
(NULL)    MECH
```

Result

INNER JOIN returns matching rows, LEFT JOIN preserves all left-table rows, and RIGHT JOIN preserves all right-table rows.

```
SQL>  SELECT * FROM Employees e INNER JOIN Departments d
      ON e.DeptId = d.DeptId;

INNER JOIN
EMP_NAME         DEPT_NAME
-------------- ------------
Anu              CSE
Bala             ECE
Charan           CSE

SQL>  SELECT * FROM Employees e LEFT JOIN Departments d
      ON e.DeptId = d.DeptId;

LEFT JOIN
EMP_NAME         DEPT_NAME
-------------- ------------
Anu              CSE
Bala             ECE
Charan           CSE
Divya            (NULL)

SQL>  SELECT * FROM Employees e RIGHT JOIN Departments d
      ON e.DeptId = d.DeptId;

RIGHT JOIN
EMP_NAME         DEPT_NAME
-------------- ------------
Anu              CSE
Charan           CSE
Bala             ECE
(NULL)           MECH

SQL>
```

6. Explicit Cursor in PL/SQL

Aim

To use an explicit cursor to retrieve and display employee names and salaries one row at a time.

Algorithm

1. Create the Employee table and insert sample records.
2. Declare an explicit cursor for the employee query.
3. Open the cursor.
4. Fetch rows into variables and display them until no rows remain.
5. Close the cursor.

SQL Program

```
SET SERVEROUTPUT ON;

CREATE TABLE Employee (
    emp_id NUMBER PRIMARY KEY,
    emp_name VARCHAR2(30),
    salary NUMBER(8,2)
);
INSERT INTO Employee VALUES (101, 'Anu', 35000);
INSERT INTO Employee VALUES (102, 'Bala', 42000);
INSERT INTO Employee VALUES (103, 'Charan', 39000);
COMMIT;

DECLARE
    CURSOR emp_cur IS
        SELECT emp_name, salary FROM Employee ORDER BY emp_id;
    v_name Employee.emp_name%TYPE;
    v_salary Employee.salary%TYPE;
BEGIN
    OPEN emp_cur;
    LOOP
        FETCH emp_cur INTO v_name, v_salary;
        EXIT WHEN emp_cur%NOTFOUND;
        DBMS_OUTPUT.PUT_LINE(v_name || ' - ' || v_salary);
    END LOOP;
    CLOSE emp_cur;
END;
/
```

Sample Output

```
Anu - 35000
Bala - 42000
Charan - 39000

PL/SQL procedure successfully completed.
```

Result

The explicit cursor fetched each employee row and displayed the employee name and salary.

```
SQL> SELECT EmpName, Salary FROM Employe

EMPNAME                         SALARY
-------------------- --------------
Anu                              35000
Bala                             42000
Charan                           39000

PL/SQL procedure successfully completed.
```

7. Trigger

Aim

To create a trigger that automatically records a message whenever a new employee is inserted.

Algorithm

1. Create the Employee table.
2. Create a Log_Table to store trigger messages.
3. Create an AFTER INSERT row-level trigger.
4. Insert an employee record.
5. Query the log table to verify that the trigger ran.

SQL Program

```
CREATE TABLE Employee (
    emp_id NUMBER PRIMARY KEY,
    emp_name VARCHAR2(30),
    salary NUMBER(8,2)
);
CREATE TABLE Log_Table (
    log_message VARCHAR2(100),
    log_date DATE
);

CREATE OR REPLACE TRIGGER trg_employee_insert
AFTER INSERT ON Employee
FOR EACH ROW
BEGIN
    INSERT INTO Log_Table(log_message, log_date)
    VALUES ('Inserted employee ID: ' || :NEW.emp_id, SYSDATE);
END;
/

INSERT INTO Employee VALUES (101, 'Anu', 35000);
COMMIT;

SELECT log_message FROM Log_Table;
```

Sample Output

```
LOG_MESSAGE
--------------------------------
Inserted employee ID: 101

1 row selected.
```

Result

The trigger automatically inserted a log message after the employee record was inserted.

```
LOG_MESSAGE
--------------------------
Inserted employee ID: 101

1 row selected.
```

8. PL/SQL Block for Exception Handling

Aim

To handle the predefined NO_DATA_FOUND exception when a query does not return a matching employee.

Algorithm

1. Create the Employee table and insert a sample record.
2. Declare a variable to store the employee name.
3. Select an employee using an ID that does not exist.
4. Handle NO_DATA_FOUND in the EXCEPTION section.
5. Display the handled message.

SQL Program

```
SET SERVEROUTPUT ON;

CREATE TABLE Employee (
    emp_id NUMBER PRIMARY KEY,
    emp_name VARCHAR2(30),
    salary NUMBER(8,2)
);
INSERT INTO Employee VALUES (101, 'Anu', 35000);
COMMIT;

DECLARE
    v_name Employee.emp_name%TYPE;
BEGIN
    SELECT emp_name INTO v_name
    FROM Employee
    WHERE emp_id = 999;

    DBMS_OUTPUT.PUT_LINE('Employee: ' || v_name);
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No employee found for the given ID.');
    WHEN TOO_MANY_ROWS THEN
        DBMS_OUTPUT.PUT_LINE('More than one employee matched.');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Unexpected error: ' || SQLERRM);
END;
/
```

Sample Output

```
No employee found for the given ID.

PL/SQL procedure successfully completed.
```

Result

The NO_DATA_FOUND exception was handled, so the block displayed a meaningful message instead of terminating with an unhandled error.

```
QL> SET SERVEROUTPUT ON;
QL> DECLARE
 2  v_dept_name departments.dept_name%TYPE;
 3  v_target_loc VARCHAR2(50) := 'Chicago';
 4  BEGIN
 5  SELECT dept_name
 6   INTO v_dept_name
 7   FROM departments
 8    WHERE location = v_target_loc;
 9  DBMS_OUTPUT.PUT_LINE('Department Name: ' || v_dept_name);
10  EXCEPTION
11    WHEN TOO_MANY_ROWS THEN
12  DBMS_OUTPUT.PUT_LINE('Error: Multiple departments found in ' || v_target_loc || '. Please use a unique Department ID instead.');
13  WHEN NO_DATA_FOUND THEN
14   DBMS_OUTPUT.PUT_LINE('Error: No departments found in ' || v_target_loc);
15  WHEN OTHERS THEN
16   DBMS_OUTPUT.PUT_LINE('Unexpected Error: ' || SQLERRM);
17  END;
18  /
rror: Multiple departments found in Chicago. Please use a unique Department ID
nstead.

L/SQL procedure successfully completed.
```

9. Stored Procedure

Aim

To create and execute a stored procedure that displays an employee's details using an employee ID.

Algorithm

1. Create the Employee table and insert sample records.
2. Create a procedure with an employee ID input parameter.
3. Select the employee name and salary into local variables.
4. Display the values; handle the case where the employee ID is not found.
5. Execute the procedure.

SQL Program

```
SET SERVEROUTPUT ON;

CREATE TABLE Employee (
    emp_id NUMBER PRIMARY KEY,
    emp_name VARCHAR2(30),
    salary NUMBER(8,2)
);
INSERT INTO Employee VALUES (101, 'Anu', 35000);
INSERT INTO Employee VALUES (102, 'Bala', 42000);
COMMIT;

CREATE OR REPLACE PROCEDURE show_employee(p_emp_id IN NUMBER) AS
    v_name Employee.emp_name%TYPE;
    v_salary Employee.salary%TYPE;
BEGIN
    SELECT emp_name, salary
    INTO v_name, v_salary
    FROM Employee
    WHERE emp_id = p_emp_id;

    DBMS_OUTPUT.PUT_LINE('Name: ' || v_name);
    DBMS_OUTPUT.PUT_LINE('Salary: ' || v_salary);
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Employee ID not found.');
END;
/

EXEC show_employee(101);
```

Sample Output

```
Name: Anu
Salary: 35000

PL/SQL procedure successfully completed.
```

Result

The stored procedure was created and executed successfully, displaying the details for employee ID 101.

```
 8  BEGIN
 9   SELECT stock_quantity
10   INTO v_current_stock
11   FROM products
12  WHERE product_id = p_prod_id;
13  IF v_current_stock < p_qty_sold THEN
14  RAISE insufficient_stock;
15   END IF;
16  UPDATE products
17   SET stock_quantity = stock_quantity - p_qty_sold
18  WHERE product_id = p_prod_id;
19    COMMIT;
20     DBMS_OUTPUT.PUT_LINE('Stock updated successfully for Product ID: ' || p_prod_id);
21  EXCEPTION
22   WHEN NO_DATA_FOUND THEN
23   DBMS_OUTPUT.PUT_LINE('Procedure Error: Product ID ' || p_prod_id || ' does not exist.');
24    WHEN insufficient_stock THEN
25    DBMS_OUTPUT.PUT_LINE('Procedure Error: Cannot sell ' || p_qty_sold || ' units. Only ' || v_current_stock || ' available.');
26    WHEN OTHERS THEN
27   DBMS_OUTPUT.PUT_LINE('Unexpected Error: ' || SQLERRM);
28  END;
29  /

rocedure created.

QL> SET SERVEROUTPUT ON;
QL> EXEC update_product_stock(101, 5);
tock updated successfully for Product ID: 101

L/SQL procedure successfully completed.

QL> EXEC update_product_stock(102, 1);
rocedure Error: Cannot sell 1 units. Only 0 available.

L/SQL procedure successfully completed.

QL> EXEC update_product_stock(999, 1);
rocedure Error: Product ID 999 does not exist.

L/SQL procedure successfully completed.
```

Activate Windows

10. Stored Function

Aim

To create and execute a stored function that returns the annual salary of an employee based on the monthly salary.

Algorithm

1. Create the Employee table and insert sample records.
2. Create a function that accepts an employee ID.
3. Retrieve the monthly salary and multiply it by 12.
4. Return the annual salary.
5. Call the function using SELECT.

SQL Program

```
CREATE TABLE Employee (
    emp_id NUMBER PRIMARY KEY,
    emp_name VARCHAR2(30),
    monthly_salary NUMBER(8,2)
);
INSERT INTO Employee VALUES (101, 'Anu', 35000);
INSERT INTO Employee VALUES (102, 'Bala', 42000);
COMMIT;

CREATE OR REPLACE FUNCTION annual_salary(p_emp_id IN NUMBER)
RETURN NUMBER
IS
    v_monthly_salary Employee.monthly_salary%TYPE;
BEGIN
    SELECT monthly_salary
    INTO v_monthly_salary
    FROM Employee
    WHERE emp_id = p_emp_id;

    RETURN v_monthly_salary * 12;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RETURN NULL;
END;
/

SELECT annual_salary(101) AS annual_salary FROM dual;
```

Sample Output

```
ANNUAL_SALARY
-------------
420000

1 row selected.
```

Result

The stored function returned the annual salary (monthly salary × 12) for employee ID 101.

```
10  BEGIN
11  IF p_discount < 0 OR p_discount > 100 THEN
12  RAISE invalid_discount;
13   END IF;
14  SELECT price
15  INTO v_original_price
16  FROM products
17   WHERE product_id = p_prod_id;
18    v_final_price := v_original_price - (v_original_price * (p_discount / 100));
19  RETURN v_final_price;
20  EXCEPTION
21   WHEN NO_DATA_FOUND THEN
22   DBMS_OUTPUT.PUT_LINE('Function Error: Product ID ' || p_prod_id || ' not found.');
23  RETURN NULL;
24   WHEN invalid_discount THEN
25  DBMS_OUTPUT.PUT_LINE('Function Error: Discount must be between 0 and 100.');
26  RETURN NULL;
27   WHEN OTHERS THEN
28   DBMS_OUTPUT.PUT_LINE('Unexpected Function Error: ' || SQLERRM);        RETURN NULL;
29   RETURN NULL;
30  END;
31  /

Function created.

annual income 420000
```

11. Hospital Management System

Aim

To maintain patient, doctor, appointment, and billing details using relational tables and SQL queries.

Algorithm

1. Create Patient and Doctor master tables.
2. Create Appointment and Bill tables with foreign keys.
3. Insert patient, doctor, appointment, and bill records.
4. Display patient and doctor appointment details using JOIN.
5. Display billing details.

SQL Program

```
CREATE TABLE H_Patient (
 patient_id NUMBER(4) PRIMARY KEY,
 patient_name VARCHAR2(40) NOT NULL,
 age NUMBER(3) CHECK (age > 0),
 phone VARCHAR2(15)
);
CREATE TABLE H_Doctor (
 doctor_id NUMBER(4) PRIMARY KEY,
 doctor_name VARCHAR2(40) NOT NULL,
 specialization VARCHAR2(30) NOT NULL
);
CREATE TABLE H_Appointment (
 appointment_id NUMBER(4) PRIMARY KEY,
 patient_id NUMBER(4) REFERENCES H_Patient(patient_id),
 doctor_id NUMBER(4) REFERENCES H_Doctor(doctor_id),
 appointment_date DATE NOT NULL
);
CREATE TABLE H_Bill (
 bill_id NUMBER(4) PRIMARY KEY,
 appointment_id NUMBER(4) REFERENCES H_Appointment(appointment_id),
 amount NUMBER(8,2) CHECK (amount >= 0),
 payment_status VARCHAR2(15) DEFAULT 'PENDING'
);

INSERT INTO H_Patient VALUES (1,'Anu',25,'9876500001');
INSERT INTO H_Patient VALUES (2,'Bala',32,'9876500002');
INSERT INTO H_Doctor VALUES (101,'Dr. Kumar','Cardiology');
INSERT INTO H_Doctor VALUES (102,'Dr. Priya','Dermatology');
INSERT INTO H_Appointment VALUES (1001,1,101,DATE '2026-11-02');
INSERT INTO H_Appointment VALUES (1002,2,102,DATE '2026-11-03');
INSERT INTO H_Bill VALUES (501,1001,1500,'PAID');
INSERT INTO H_Bill (bill_id,appointment_id,amount)
VALUES (502,1002,800);
COMMIT;

SELECT a.appointment_id,p.patient_name,d.doctor_name,
       d.specialization,a.appointment_date
FROM H_Appointment a
JOIN H_Patient p ON a.patient_id=p.patient_id
JOIN H_Doctor d ON a.doctor_id=d.doctor_id
ORDER BY a.appointment_id;

SELECT * FROM H_Bill ORDER BY bill_id;
```

Sample Output

```
APPOINTMENT_ID  PATIENT_NAME  DOCTOR_NAME  SPECIALIZATION  APPOINTMENT_DATE
1001            Anu           Dr. Kumar    Cardiology      02-NOV-26
1002            Bala          Dr. Priya    Dermatology     03-NOV-26

BILL_ID  APPOINTMENT_ID  AMOUNT  PAYMENT_STATUS
501      1001            1500    PAID
502      1002            800     PENDING
```

Result

The Hospital Management System tables were created and populated. Appointment and billing details were successfully retrieved using SQL joins and SELECT queries.

SQL Plus

```
SQL> SELECT * FROM Appointments;

APPOINTMENT_ID  PATIENT_NAME  DOCTOR_NAME  SPECIALIZATION  APPOINTMENT_DATE
--------------  ------------  -----------  --------------  --------------
1001            Anu           Dr. Kumar    Cardiology      02-NOV-26
1002            Bala          Dr. Priya    Dermatology     03-NOV-26

SQL> SELECT * FROM Bills;

BILL_ID  APPOINTMENT_ID  AMOUNT  PAYMENT_STATUS
-------  --------------  ------  --------------
501      1001            1500    PAID
502      1002            800     PENDING
```

12. Hotel Management System

Aim

To store hotel room, guest, booking, and payment details and retrieve room availability and booking information.

Algorithm

1. Create Room, Guest, Booking, and Payment tables.
2. Apply primary key, foreign key, UNIQUE, and CHECK constraints.
3. Insert room and guest details.
4. Insert booking and payment records.
5. Display available rooms and booking details using JOIN.

SQL Program

```
CREATE TABLE Hotel_Room (
 room_id NUMBER(4) PRIMARY KEY,
 room_type VARCHAR2(20) NOT NULL,
 room_rate NUMBER(8,2) CHECK (room_rate > 0),
 room_status VARCHAR2(15) DEFAULT 'AVAILABLE'
);
CREATE TABLE Hotel_Guest (
 guest_id NUMBER(4) PRIMARY KEY,
 guest_name VARCHAR2(40) NOT NULL,
 phone VARCHAR2(15) UNIQUE
);
CREATE TABLE Hotel_Booking (
 booking_id NUMBER(4) PRIMARY KEY,
 guest_id NUMBER(4) REFERENCES Hotel_Guest(guest_id),
 room_id NUMBER(4) REFERENCES Hotel_Room(room_id),
 check_in DATE NOT NULL,
 check_out DATE NOT NULL,
 CONSTRAINT chk_hotel_dates CHECK (check_out > check_in)
);
CREATE TABLE Hotel_Payment (
 payment_id NUMBER(4) PRIMARY KEY,
 booking_id NUMBER(4) REFERENCES Hotel_Booking(booking_id),
 amount NUMBER(8,2) CHECK (amount >= 0),
 payment_mode VARCHAR2(15)
);

INSERT INTO Hotel_Room VALUES (101,'Single',1500,'AVAILABLE');
INSERT INTO Hotel_Room VALUES (102,'Double',2500,'BOOKED');
INSERT INTO Hotel_Room VALUES (103,'Deluxe',4000,'AVAILABLE');
INSERT INTO Hotel_Guest VALUES (1,'Anu','9876500001');
INSERT INTO Hotel_Guest VALUES (2,'Bala','9876500002');
INSERT INTO Hotel_Booking VALUES
(201,1,102,DATE '2026-11-05',DATE '2026-11-07');
INSERT INTO Hotel_Booking VALUES
(202,2,101,DATE '2026-11-06',DATE '2026-11-08');
INSERT INTO Hotel_Payment VALUES (301,201,5000,'UPI');
INSERT INTO Hotel_Payment VALUES (302,202,3000,'CARD');
COMMIT;


SELECT room_id,room_type,room_rate FROM Hotel_Room
WHERE room_status='AVAILABLE' ORDER BY room_id;
SELECT b.booking_id,g.guest_name,r.room_id,r.room_type,
       b.check_in,b.check_out
FROM Hotel_Booking b
JOIN Hotel_Guest g ON b.guest_id=g.guest_id
JOIN Hotel_Room r ON b.room_id=r.room_id
```

```
ORDER BY b.booking_id;
SELECT * FROM Hotel_Payment ORDER BY payment_id;
```

Sample Output

```
AVAILABLE ROOMS
ROOM_ID  ROOM_TYPE  ROOM_RATE
101      Single     1500
103      Deluxe     4000

BOOKING_ID  GUEST_NAME  ROOM_ID  ROOM_TYPE
201         Anu         102      Double
202         Bala        101      Single

PAYMENT_ID  BOOKING_ID  AMOUNT  PAYMENT_MODE
301         201         5000    UPI
302         202         3000    CARD
```

Result

The Hotel Management System was created successfully. Room availability, guest bookings, and payment information were stored and displayed.

SQL Plus

```
AVAILABLE ROOMS
-----------------------------------------
ROOM_ID   ROOM_TYPE                ROOM_RATE
-----------------------------------------
101       Single                       1500
103       Deluxe                       4000

BOOKING_ID  GUEST_NAME   ROOM_ID  ROOM_TYPE
-----------------------------------------
201         Anu          102      Double
202         Bala         101      Single

PAYMENT_ID  BOOKING_ID  AMOUNT   PAYMENT_MODE
-----------------------------------------
301         201         5000     UPI
302         202         3000     CARD

SQL>
```

13. Timetable Management System

Aim

To store department, faculty, subject, and class timetable details and display a class timetable in day and period order.

Algorithm

1. Create Department, Faculty, Subject, and Class_Timetable tables.
2. Add primary key and foreign key constraints.
3. Insert department, faculty, and subject data.
4. Insert timetable entries for a class.
5. Join the tables to display the timetable and faculty names.

SQL Program

```
CREATE TABLE T_Department (
 dept_id NUMBER(3) PRIMARY KEY,
 dept_name VARCHAR2(20) UNIQUE NOT NULL
);
CREATE TABLE T_Faculty (
 faculty_id NUMBER(4) PRIMARY KEY,
 faculty_name VARCHAR2(40) NOT NULL,
 dept_id NUMBER(3) REFERENCES T_Department(dept_id)
);
CREATE TABLE T_Subject (
 subject_id NUMBER(4) PRIMARY KEY,
 subject_name VARCHAR2(40) NOT NULL,
 dept_id NUMBER(3) REFERENCES T_Department(dept_id)
);
CREATE TABLE Class_Timetable (
 timetable_id NUMBER(4) PRIMARY KEY,
 class_name VARCHAR2(15) NOT NULL,
 day_name VARCHAR2(10) NOT NULL,
 period_no NUMBER(2) CHECK (period_no BETWEEN 1 AND 8),
 subject_id NUMBER(4) REFERENCES T_Subject(subject_id),
 faculty_id NUMBER(4) REFERENCES T_Faculty(faculty_id),
 CONSTRAINT uq_class_slot UNIQUE(class_name,day_name,period_no)
);

INSERT INTO T_Department VALUES (10,'CSE');
INSERT INTO T_Faculty VALUES (1,'Mr. Arun',10);
INSERT INTO T_Faculty VALUES (2,'Ms. Divya',10);
INSERT INTO T_Subject VALUES (101,'DBMS',10);
INSERT INTO T_Subject VALUES (102,'Computer Networks',10);
INSERT INTO T_Subject VALUES (103,'Java',10);
INSERT INTO Class_Timetable VALUES (1,'CSE-A','MONDAY',1,101,1);
INSERT INTO Class_Timetable VALUES (2,'CSE-A','MONDAY',2,102,2);
INSERT INTO Class_Timetable VALUES (3,'CSE-A','TUESDAY',1,103,1);
INSERT INTO Class_Timetable VALUES (4,'CSE-A','TUESDAY',2,101,1);
INSERT INTO Class_Timetable VALUES (5,'CSE-A','WEDNESDAY',1,102,2);
COMMIT;

SELECT t.class_name,t.day_name,t.period_no,
       s.subject_name,f.faculty_name
FROM Class_Timetable t
JOIN T_Subject s ON t.subject_id=s.subject_id
JOIN T_Faculty f ON t.faculty_id=f.faculty_id
WHERE t.class_name='CSE-A'
ORDER BY CASE t.day_name
 WHEN 'MONDAY' THEN 1 WHEN 'TUESDAY' THEN 2
 WHEN 'WEDNESDAY' THEN 3 WHEN 'THURSDAY' THEN 4
```

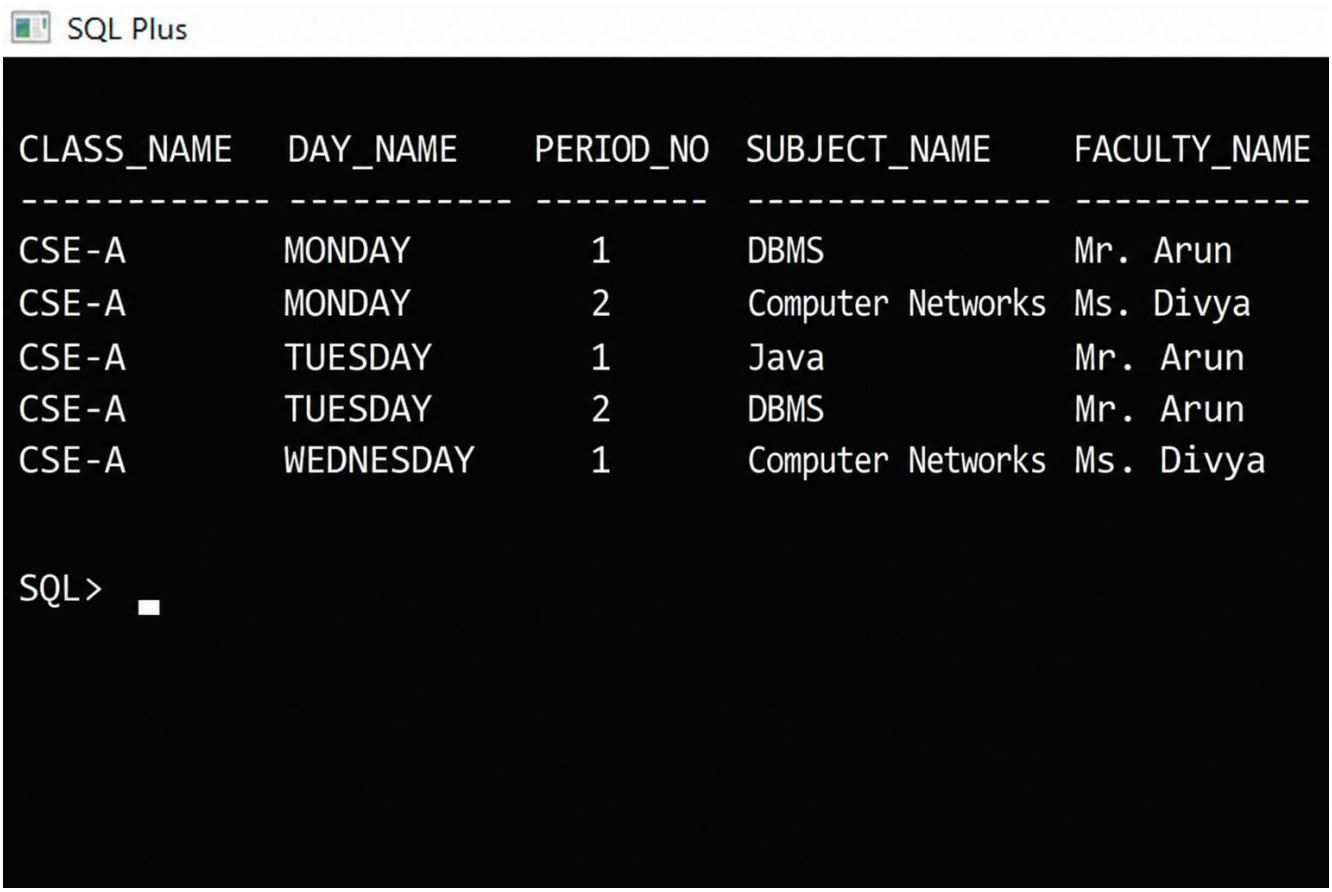
```
 WHEN 'FRIDAY' THEN 5 WHEN 'SATURDAY' THEN 6 ELSE 7
END,t.period_no;
```

Sample Output

```
CLASS_NAME  DAY_NAME   PERIOD_NO  SUBJECT_NAME       FACULTY_NAME
CSE-A       MONDAY     1          DBMS               Mr. Arun
CSE-A       MONDAY     2          Computer Networks  Ms. Divya
CSE-A       TUESDAY    1          Java               Mr. Arun
CSE-A       TUESDAY    2          DBMS               Mr. Arun
CSE-A       WEDNESDAY  1          Computer Networks  Ms. Divya
```

Result

The Timetable Management System was created successfully. Subject and faculty schedules were stored and retrieved for the selected class.



14. Railway Reservation System

Aim

To maintain train, passenger, reservation, and payment information and retrieve ticket details using SQL queries.

Algorithm

1. Create Train, Passenger, Reservation, and Payment tables.
2. Define primary key, foreign key, and CHECK constraints.
3. Insert train and passenger records.
4. Insert reservation and payment records.
5. Display ticket details by joining passenger, train, and reservation tables.
6. Count reservations for each train.

SQL Program

```
CREATE TABLE R_Train (
 train_no NUMBER(5) PRIMARY KEY,
 train_name VARCHAR2(40) NOT NULL,
 source_station VARCHAR2(30) NOT NULL,
 destination_station VARCHAR2(30) NOT NULL
);
CREATE TABLE R_Passenger (
 passenger_id NUMBER(5) PRIMARY KEY,
 passenger_name VARCHAR2(40) NOT NULL,
 age NUMBER(3) CHECK (age > 0),
 gender VARCHAR2(10)
);
CREATE TABLE R_Reservation (
 pnr_no NUMBER(8) PRIMARY KEY,
 passenger_id NUMBER(5) REFERENCES R_Passenger(passenger_id),
 train_no NUMBER(5) REFERENCES R_Train(train_no),
 journey_date DATE NOT NULL,
 seat_no NUMBER(4),
 booking_status VARCHAR2(15) DEFAULT 'CONFIRMED',
 CONSTRAINT chk_booking_status CHECK
 (booking_status IN ('CONFIRMED','WAITLIST','CANCELLED'))
);
CREATE TABLE Train_Payment (
 payment_id NUMBER(5) PRIMARY KEY,
 pnr_no NUMBER(8) REFERENCES R_Reservation(pnr_no),
 amount NUMBER(8,2) CHECK (amount >= 0),
 payment_status VARCHAR2(15)
);

INSERT INTO R_Train VALUES
(12601,'Chennai Express','Chennai','Mumbai');
INSERT INTO R_Train VALUES
(12602,'Coimbatore Express','Chennai','Coimbatore');
INSERT INTO R_Passenger VALUES (1,'Anu',25,'Female');
INSERT INTO R_Passenger VALUES (2,'Bala',30,'Male');
INSERT INTO R_Reservation VALUES
(700001,1,12601,DATE '2026-11-10',21,'CONFIRMED');
INSERT INTO R_Reservation VALUES
(700002,2,12602,DATE '2026-11-11',15,'WAITLIST');
INSERT INTO Train_Payment VALUES (901,700001,1200,'PAID');
INSERT INTO Train_Payment VALUES (902,700002,800,'PENDING');
COMMIT;

SELECT r.pnr_no,p.passenger_name,t.train_name,
       t.source_station,t.destination_station,
       r.journey_date,r.seat_no,r.booking_status
```

```
FROM R_Reservation r,
JOIN R_Passenger p ON r.passenger_id=p.passenger_id
JOIN R_Train t ON r.train_no=t.train_no
ORDER BY r.pnr_no;

SELECT * FROM Train_Payment ORDER BY payment_id;

SELECT t.train_name,COUNT(r.pnr_no) AS total_reservations
FROM R_Train t
LEFT JOIN R_Reservation r ON t.train_no=r.train_no
GROUP BY t.train_name ORDER BY t.train_name;
```

Sample Output

```
PNR_NO  PASSENGER_NAME  TRAIN_NAME          SOURCE   DESTINATION  SEAT_NO  BOOKING_STATUS
700001  Anu             Chennai Express     Chennai  Mumbai       21       CONFIRMED
700002  Bala            Coimbatore Express  Chennai  Coimbatore   15       WAITLIST

PAYMENT_ID  PNR_NO  AMOUNT  PAYMENT_STATUS
901         700001  1200    PAID
902         700002  800     PENDING

TRAIN_NAME          TOTAL_RESERVATIONS
Chennai Express     1
Coimbatore Express  1
```

Result

The Railway Reservation System was created successfully. Train, passenger, reservation, and payment details were stored and retrieved. The reservation count for each train was also displayed.

